

Architecture

Early Access · MessageFoundry v0.3.2 · September 2026

Architectural standard: modular, contract-bounded components

MessageFoundry uses independent components with defined interfaces. Each component hides its internal design, so teams can work in parallel and limit conflicts. This standard guides the topology, store, and module map below.

Modular design (component-based architecture). The independent components are the headless **engine**, **configuration**, **web console** (`/ui`), **IDE extension**, **PySide6 test harness**, and **CLI tools**. Configuration contains Connections, Routers, Handlers, and environment values. The CLI includes `generate`, `check`, and `dryrun`, in addition to `serve`.

NSSM runs the Windows service, and CI checks changes. Engine packages are `config`, `parsing`, `store`, `transports`, `pipeline`, `auth`, and `api`. These boundaries limit the code that each change or AI context must include.

Information hiding (Parnas). David Parnas described this principle in his 1972 paper "*On the Criteria To Be Used in Decomposing Systems into Modules*." Each module hides a design decision behind an interface. Teams can then work on separate modules, and internal changes need not affect other components.

Separation of concerns: high cohesion and loose coupling.

- **High cohesion** keeps one concern inside one component, such as message processing in the engine or the interface in the web console.
- **Loose coupling** limits dependencies to defined interfaces. A change in one component then affects fewer other components.
- Smaller shared interfaces reduce conflicts between teams that work at the same time.

Contract-first / interface-driven design. Define each interface before building the components on either side. Each team then implements its component independently. In this repo: - the **HTTP/WebSocket API** (`api/app.py`) connects the engine and its clients (web console, IDE, harness). - the **connector registry** (`transports/base.py`) defines the interface for pluggable transports. - the **one-way dependency rule** keeps `pipeline` / `transports` / `parsing` / `store` / `config` from importing `api`.

Team organization.

- **Conway's Law** describes how system structure can reflect communication between teams. The **Inverse Conway Maneuver** defines component boundaries to support the intended division of work. See `WORKTREES.md`.

- Each component has one owner at a time. Domain-driven design calls this a **bounded context**. Concurrent sessions keep changes in their own module and branch.

Topology: engine-as-library + localhost API

The engine is an importable Python package (`messagefoundry`). Clients — primarily the **browser web console** (`/ui` , `messagefoundry_webconsole`) — drive it over a localhost HTTP + WebSocket API (FastAPI/uvicorn). The same API serves three deployments without code changes:

- **Embedded** — another Python program owns the engine in-process via `create_app(engine)` (the async test client, embedding). A UI client runs in a separate process or browser. It uses only the HTTP API and never imports the engine.
- **Local daemon** — engine runs as a Windows service / Linux daemon. The web console attaches over the API. See `SERVICE.md` for the Windows service setup (NSSM).
- **Remote** — the same API over the network. Remote access is supported and **off by default**. Set `[security].local_access_only = false` . Set the bind address in `[security].listen_address` . Configure in-process TLS through `[api].tls_cert_file` , or use a declared trusted proxy. The proxy settings are `[api].tls_terminated_upstream` and `[api].trusted_proxies` . Auth is always required, and an off-loopback plaintext bind is refused at startup.

The library API defines the logical boundary. Deployment determines whether components share a process or communicate across processes.

The diagram shows separate clients connected through the API. Engine packages never import `api` :

```
flowchart TB
    classDef client fill:#e3f2fd,stroke:#1565c0,color:#0d2b45;
    classDef api fill:#ede7f6,stroke:#5e35b1,color:#22103f;
    classDef engine fill:#e8f5e9,stroke:#2e7d32,color:#10240f;
    classDef deploy fill:#eceff1,stroke:#546e7a,color:#1c2429;

    CON["Web console /ui<br/>(browser)"]:::client
    IDE["VS Code extension"]:::client
    HARNESS["Test harness<br/>(PySide6)"]:::client

    subgraph API_BND["API — localhost 127.0.0.1 · auth + RBAC · the only external surface"]
        API["api/ — FastAPI + uvicorn<br/>HTTP + WebSocket"]:::api
        AUTH["auth/ — authn + RBAC<br/>deny-by-default · hash-chained audit"]:::api
    end

    subgraph ENGINE["Engine — headless asyncio service (no GUI imports)"]
        PIPE["pipeline/ — RegistryRunner<br/>listener · router · transform · delivery workers"]:::engine
        TRANS["transports/ — connector registry<br/>MLLP · TCP · HTTP · File · SFTP/FTP · REST · SOAP · Database · X12 · FHIR · DICOM · Email · ..."]:::engine
        PARSE["parsing/ — pure HL7/X12/DICOM library<br/>python-hl7 · hl7apy · X12 codec · DICOM codec · base64 binary codec"]:::engine
        STORE["store/ — staged queue<br/>SQLite WAL · SQL Server · AES-256-GCM"]:::engine
        CFG["config/ — code-first wiring<br/>Connections · Routers · Handlers · environments/"]:::engine
    end
```

```

NSSM["NSSM Windows service<br/>(messagefoundry serve)"]:::deploy

CON -->|"HTTP/WS API client"| API
IDE -->|"HTTP"| API
HARNESS -->|"MLLP send/receive"| TRANS
CON -->|"may import (pure lib)"| PARSE

API --> AUTH
API ==>|"depends on engine"| PIPE

PIPE --> TRANS
PIPE --> PARSE
PIPE --> STORE
PIPE --> CFG
TRANS --> PARSE

NSSM ==> ENGINE

```

The message store *is* the queue

The store uses a transactional **staged queue** on SQLite in WAL mode. One generic `queue` table has a `stage` discriminator: `ingress`, `routed`, or `outbound`. **Step B** separates routing from transformation (ADR 0001 — [docs/adr/0001-staged-pipeline-architecture.md](https://docs.adrs.com/adr/0001-staged-pipeline-architecture.md)):

```

inbound msg → decode / parse / (strict-validate)  [synchronous – still NAK on failure]
|
▼ persist raw to the INGRESS stage → commit → ACK source  [ACK-on-receipt]
|   (ingress + routed lanes key on channel_id; outbound on destination_name)
▼ router worker (per inbound): run Router
produce one ROUTED row per selected handler + complete the ingress row  (one txn)
|
▼ transform worker (per inbound): run that handler's transform
produce an OUTBOUND row per destination + complete the routed row  (one txn)
|
▼ per-destination delivery worker
deliver → mark done | mark failed+reschedule (retry policy)

```

The engine sends an **ACK on receipt**, after it commits the raw message to the ingress stage. Routing, transformation, and delivery follow that commit. A slow Router, transform, or outbound therefore does not stop intake.

Each stage handoff uses one committed transaction: claim the row, produce the next-stage rows, and complete the current row. A crash before commit rolls back the transaction for another attempt. Each handoff is idempotent, and `reset_stale_inflight` recovers in-flight rows from **every** stage at startup.

The queue supports **at-least-once delivery**, **retries**, and **replay** without a separate broker. **Routers and transforms must be pure** so another attempt produces the same output. **Outbound connections must be idempotent**. The store preserves correlation IDs for duplicate detection. Each destination drains independently, and a slow transform does not stop routing.

The **store finalizer** sets the final message disposition. It records **RECEIVED** at ingress and **ROUTED** or **UNROUTED** after the Router. When no work remains in flight, it selects:

- **PROCESSED** : all delivery work completed.
- **FILTERED** : every Handler ran but delivered nothing.
- **NOT_DEPLOYED** : all addressed destinations are in the graph with **deployed=false** .
- **ERROR** : a stage sent the message to the dead-letter path.

The finalizer waits while an earlier-stage row remains pending. One completed Handler cannot finalize a message while another Handler still awaits transformation.

The ACK confirms receipt and persistence only. Post-ACK routing or transform failures produce logs, dead-letter records, and AlertSink notifications. They do not send a NAK to the source.

NOT_DEPLOYED (BACKLOG #233, **ADR 0111**) requires a separate event record. The finalizer selects **FILTERED** when no queue rows remain and the message is still **ROUTED** . Without that record, a declined destination would appear as a filtered message.

Before an outbound-stage row is created, the transform worker declines each **Send** to an outbound with **deployed=false** . It writes one **not_deployed message_events** row per declined destination in the same **MessageStore.transform_handoff** transaction.

If no deliveries occur, that event produces **NOT_DEPLOYED** . Without the event, the result is **FILTERED** . If a deployed sibling delivers, the result is **PROCESSED** , and the event still records the skipped destination. The **message_events** verbosity setting cannot remove this event because the final disposition requires it.

Dry-run and the Test Bench do not run the finalizer. They therefore show a fully declined message as **FILTERED** . Declined names remain in **RouteOutcome.declined** and do not pass into **DryRunResult** .

The router/transform split was taken after Step A's measured write amplification (now ~3 durable transactions/message for a single-handler message, +1 per extra handler) — recorded in **docs/benchmarks/step-b-write-amplification.md**. An optional per-inbound **ack_after=delivered** (defer the ACK until delivery succeeds) is planned but not built — the pipeline is ACK-on-receipt only.

The same staged flow as a rendered diagram:

flowchart TB

```
classDef stage fill:#fff3e0,stroke:#ef6c00,color:#3a1d00;
classDef worker fill:#e8f5e9,stroke:#2e7d32,color:#10240f;
classDef disp fill:#ede7f6,stroke:#5e35b1,color:#22103f;
classDef io fill:#e3f2fd,stroke:#1565c0,color:#0d2b45;

SRC(["Inbound connection<br/>MLLP / File"]>:::io
LISTEN["Listener<br/>decode · parse · (strict-validate)"]:::worker
NAK["NAK (AR/AE) + ERROR<br/>synchronous, pre-ingress"]:::disp

ING(["ingress stage<br/>raw committed"]>:::stage
ACK(["ACK (AA) – on receipt"]>:::io
RW["Router worker (per inbound)<br/>run @router – pure"]:::worker
```

```

ROUTED[("routed stage<br/>one row per selected handler")]:::stage
TW["Transform worker (per inbound)<br/>run @handler transform - pure"]:::worker
OUT[("outbound stage<br/>one row per destination")]:::stage
DW["Delivery worker (per outbound)<br/>idempotent send · retry · dead-letter"]:::worker
DEST[("Outbound connection(s)")]:::io

FIN{"Store finalizer<br/>single disposition authority"}:::disp
D1["RECEIVED"]:::disp
D2["ROUTED / UNROUTED"]:::disp
D3["PROCESSED / FILTERED<br/>NOT_DEPLOYED / ERROR"]:::disp

SRC --> LISTEN
LISTEN -->|"decode/parse/validate fail"| NAK
LISTEN -->|"ok"| ING
ING --> ACK
ING ==>|"committed txn"| RW
RW ==>|"committed txn"| ROUTED
ROUTED ==>|"committed txn"| TW
TW ==>|"committed txn"| OUT
OUT --> DW
DW --> DEST

ING -->|"records"| D1
RW -->|"records"| D2
DW -->|"records"| D3
D1 --> FIN
D2 --> FIN
D3 --> FIN

```

Concurrency

The asyncio runtime has one listener, Router worker, and transform worker per **inbound connection**. Each **outbound connection** has one delivery worker. `RegistryRunner` supervises listeners, file/database pollers, Router/transform workers, and retry timers.

It restarts a worker after a crash. Router and transform workers continue with acknowledged messages when the source stops. Each worker class has its own wake event. A producer wakes only its downstream consumer, which prevents cross-class lost wakeups.

Startup wiring also isolates connection failures (ADR 0031). Failures include an unresolved `env()` value or certificate, an egress refusal, an occupied port, or refused cleartext exposure. The engine records the connection as `failed`, logs the failure, and sends an alert. It starts the remaining graph and serves the API.

A failed outbound still has a delivery worker. The worker retries queued messages, and a successful reload or restart restores the connection. A failed inbound does not listen. Reload remains fail-fast: it uses `build_check` on the whole new registry before it stops a healthy graph. Only startup permits partial operation.

Parsing: tolerant-first, strict-on-demand

- `python-h17` parses tolerantly and powers field *peek* for routing. Hot path.

- **hl7apy** does version-aware + profile validation, opt-in per inbound connection (`validation.strict`). Slower. Kept off the routing hot path.

Real-world HL7 v2 is frequently non-conformant. Strict-by-default would drop messages that must still route, so tolerance is the default and strictness is an opt-in.

Configuration: connections + code-first routing

The target model is a **graph wired by name, authored as Python** — no enclosing "channel" object:

- a **Connection** is a named inbound or outbound endpoint (MLLP, file, ...).
- an inbound Connection names a **Router** — a Python script that sees each received message. It selects **Handlers** or filters the message.
- a **Handler** is a Python script that filters → transforms → sends to outbound Connection(s).

Configuration is version-controlled Python. The database stores runtime state and messages **only**, never configuration.

Connections/Routers/Handlers are authored against the `messagefoundry` surface (`inbound / outbound / @router / @handler / Send / MLLP / File / Message`). A directory of such modules loads via `load_config` into a `Registry` that the engine's `RegistryRunner` runs.

The configuration graph wired by name (no enclosing "channel" object) as a rendered diagram:

```
flowchart LR
    classDef conn fill:#e3f2fd,stroke:#1565c0,color:#0d2b45;
    classDef router fill:#fff3e0,stroke:#ef6c00,color:#3a1d00;
    classDef handler fill:#e8f5e9,stroke:#2e7d32,color:#10240f;

    IB["inbound: IB_ACME_ADT<br/>(MLLP)"]:::conn
    R(["@router<br/>sees every message · filters · forwards by name"]):::router
    H1["@handler: to_EHR<br/>filter → transform"]:::handler
    H2["@handler: to_archive<br/>filter → transform"]:::handler
    OB1["outbound: OB_EHR_ADT<br/>(MLLP)"]:::conn
    OB2["outbound: OB_ARCHIVE<br/>(File)"]:::conn

    IB -->|"names a router"| R
    R -->|"forward to handler(s)"| H1
    R --> H2
    H1 -->|"Send → outbound"| OB1
    H2 -->|"Send → outbound"| OB2
```

PHI / security

Messages contain PHI. Access control and the *data* protections are tracked separately — see `SECURITY.md` (identity, RBAC, action audit) and `PHI.md` (data-at-rest, transport, logging, retention, de-identification), which is the authoritative built-vs-planned map. The per-interface trust boundaries and STRIDE threats are in the internal

security/THREAT-MODEL.md (PW.1–2 / ASVS V15). [SECURITY-DOCS-POLICY.md](#) explains withheld material and access requests.

Built controls include authentication, role-based access, and a hash-chained, append-only audit log. PHI views require `messages:view_raw` or `messages:view_summary`. Audit records identify users who view, search, or replay messages.

When a key is set, the store cipher encrypts message bodies with AES-256-GCM. Owner-only database/WAL file permissions and required volume encryption protect the remaining stored data.

Additional controls are documented in [PHI.md](#). PHI redaction uses standard-library `logging`, not structlog. An always-on handler filter and `safe_exc()` remove HL7-shaped content from logged exceptions and stored dispositions.

Per-connection MLLP-over-TLS supports TLS 1.2+, server-certificate verification, hostname checks, and optional mTLS. Startup rejects off-loopback plaintext listeners. The async `RetentionRunner` enforces retention and purge rules.

Module map

Package / module	Responsibility
<code>messagefoundry.config</code>	Connector models (<code>models.py</code>) + code-first wiring registry/loader (<code>wiring.py</code>) + service settings (<code>settings.py</code>)
<code>messagefoundry.parsing</code>	Tolerant peek (python-hl7) + strict validate (hl7apy). Parse tree (<code>tree.py</code>) and the <code>Message</code> transform model (<code>message.py</code>). Pure non-HL7 codecs — X12 EDI (<code>x12/</code>), DICOM headers/SR over pydicom (<code>dicom/</code> , ADR 0025), and the base64 binary-carriage codec (<code>binary.py</code> , ADR 0028)
<code>messagefoundry.store</code>	Durable message store / staged queue (one <code>queue</code> table, <code>stage</code> = ingress routed outbound), SQLite WAL. Every receipt logged with a disposition that flows with the message. <code>Store</code> protocol + <code>open_store</code> factory in <code>base.py</code> . Production server-DB backends in <code>postgres.py</code> and <code>sqlserver.py</code>
<code>messagefoundry.transports</code>	Inbound & outbound connections (MLLP, file, X12 raw-TCP, DICOM C-STORE SCP inbound over pynetdicom — ADR 0025, ...), resolved through a registry (<code>base.py</code>) — never special-cased in <code>pipeline/</code>
<code>messagefoundry.anon</code>	Deterministic, secret-per-dataset pseudonymization / de-identification (fail-closed, ADR 0030) — exposed to the tee (<code>anonymize-captures</code>) and the test harness
<code>messagefoundry.pipeline</code>	Per-message routing/handling (<code>RegistryRunner</code> in <code>wiring_runner.py</code>) + per-inbound-connection supervision (<code>engine.py</code>). Offline <code>dryrun.py</code>
<code>messagefoundry.api</code>	Localhost FastAPI surface (<code>app.py</code> + response <code>models.py</code>) — the engine's only external interface. The <code>/ui</code> browser web console mounts onto this app from a separate wheel (see below) via <code>mount_ui</code> , pinned against the <code>api/_ui_seam.py</code> contract (WEBCONSOLE-PACKAGE.md)

Package / module	Responsibility
<code>messagefoundry.apiclient</code>	Qt-free / FastAPI-free engine-client library (ADR 0088) — a small typed httpx wrapper over the API, shared by the test harness and any future client
<code>messagefoundry.auth</code>	Authn + RBAC core (no FastAPI): permissions/roles, <code>Identity</code> , password hashing, opaque session tokens, LDAP/Kerberos (<code>service.py</code>) — enforced by <code>api</code>
<code>messagefoundry.generators</code>	Conformant synthetic HL7 generators (ADT, ORM, ORU, ...) behind <code>messagefoundry generate</code>
<code>messagefoundry.service</code>	Local Windows service control (<code>messagefoundry service {install,start,stop,status}</code> . ADR 0088), wrapping the NSSM scripts
<code>__main__.py</code> , <code>checks.py</code>	CLI entrypoint (<code>serve</code> / <code>generate</code> / <code>check</code> / <code>service</code>) + the <code>check</code> commit/CI gate

Dependency direction is one-way: `pipeline` / `transports` / `parsing` / `store` / `config` never import `api` . The API depends on the engine. The clients (web console, harness) depend on the API (via `apiclient`). The former PySide6 desktop console (`messagefoundry.console`) was retired — its reusable Qt widgets moved to the harness (BACKLOG #103, ADR 0032 retired).

Separate components outside `messagefoundry.*` :

- Operator configuration: `--config` modules and `environments/` overrides.
- Web console: `messagefoundry_webconsole/` , in the separately versioned `messagefoundry-webconsole` wheel. It mounts same-origin under `/ui` (ADR 0065, [WEBCONSOLE-PACKAGE.md](#)).
- IDE extension: `ide/` .
- Test harness: `harness/` .
- Sample configuration and messages: `samples/` .
- Windows service scripts: `scripts/service/` .
- CI and supply-chain workflows: `.github/workflows/` .

Dependencies

Declared in `pyproject.toml` (the source of truth) as `>=` minimums. The pinned, hashed resolution lives in the committed `uv.lock` / `requirements.lock` (DEP-1, see [SECURITY.md](#)). Requires **Python 3.14+**.

Runtime

- `hl7` (python-hl7) — fast, tolerant parsing for the routing hot path
- `hl7apy` — version-aware validation + profiles (opt-in strict path)
- `pydantic` — config / connector models and validation
- `aiosqlite` — async SQLite for the message store / queue
- `fastapi` + `uvicorn[standard]` — localhost engine API
- `argon2-cffi` — argon2id password hashing
- `cryptography` — AES-256-GCM at-rest encryption for the store

- `ldap3` + `pyspnego` — Active Directory / LDAP auth and Windows SSO (Kerberos)

Optional extras

- `harness` → `PySide6` only (LGPL — chosen so the OSS test harness GUI is distributable, not PyQt). The harness is the sole PySide6 consumer (was `[console]` before the desktop console was retired — BACKLOG #103. Its HTTP client, `httpx` + `truststore`, is now a base runtime dependency rather than part of this extra)
- `sqlserver` → `aiodbc` (production SQL Server store, also needs the OS-level Microsoft ODBC Driver 18 for SQL Server, which is not pip-installable, lazy-imported so SQLite-only installs skip it)
- `dicom` → `pydicom` + `pynetdicom` (DICOM codec — headers/SR only, no numpy — and the C-STORE SCP inbound connector, ADR 0025, lazy-imported so non-DICOM installs skip it)
- `dev` → `pytest` (with the `asyncio`, `timeout` and `rerunfailures` plugins), `ruff`, `mypy` (`httpx`, the ASGI test client, is now a base dependency and is no longer declared here)

Build / tooling — `hatchling` (build backend), Ruff (format + lint, no Black), mypy (strict), pytest.

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.